

Candidate Projects and Selection

Career objective shaping the choice

The student is preparing for a software-engineering role at frontier AI labs (e.g., Anthropic, Google DeepMind), specifically on agent-tooling and product engineering teams (think: Claude Code, Code Interpreter, agent infrastructure). The project should produce an artifact that doubles as portfolio evidence of "I can build production-grade infrastructure around LLMs and evaluate it rigorously."

Background-shaped constraints

- Solid general SWE background; light hands-on ML.
- Comfortable with Docker; light Linux internals exposure.
- Cloud/lab compute budget available (~\$200–\$400 envelope).
- Risk preference: solid execution within known evaluation methodology.

These constraints rule out projects that depend on model training, fine-tuning, or research-grade ML methodology — not because the student couldn't learn it, but because compressing "learn ML internals deeply" *and* "produce a defensible artifact" into 14 weeks is the textbook way to miss both targets.

The three candidates

Candidate A — Domain-specialized coding agent with a self-built benchmark

One-line. Build an LLM agent specialized for one narrow coding task family (e.g., adding type hints to existing Python code, migrating Flask endpoints to FastAPI, generating Airflow DAGs from natural-language specifications) plus a 40–60 task hand-curated benchmark with deterministic graders, then compare 2–3 frontier models on it.

Artifact. Agent harness + benchmark suite + leaderboard + failure-mode analysis.

Evaluation methodology. Borrowed from existing coding-agent benchmarks (SWE-bench, HumanEval, MBPP) — pass@k against deterministic graders.

Candidate B — Multi-model agent router with cost/quality/latency optimization

One-line. Build a middleware layer that sits between an agent and multiple LLM providers, doing prompt caching + semantic routing + cost-aware retries. Evaluate cost-vs-quality tradeoffs on an existing benchmark.

Artifact. Routing service + integration with an existing benchmark + cost/quality Pareto curves across configurations.

Evaluation methodology. Reuses an existing public benchmark (e.g., GAIA, AgentBench, ToolBench) — the contribution is the optimization layer, not the eval design.

Candidate C — Hardened, threat-modeled Python execution sandbox for LLM agents

One-line. Build a Python-only execution service with two progressively stronger isolation back-ends (hardened Docker, then gVisor) and a curated adversarial test suite that measures isolation strength empirically. Compare back-ends on isolation vs. performance.

Artifact. Sandbox service + threat model + functional test suite + adversarial test suite (the unique contribution) + performance benchmark + minimal reference agent demo.

Evaluation methodology. Constructed by the student — multi-dimensional eval (functional correctness, isolation strength under adversarial inputs, performance overhead), reported under each back-end with confidence intervals.

Comparison matrix

Dimension	A. Coding-agent + benchmark	B. Multi-model router	C. Hardened sandbox
Feasibility in 14 weeks (solo)	High — bounded, can scale benchmark size down if needed	High — modular; can ship a thin slice	Medium-high — bounded if scope held to Python-only; risk of scope creep on isolation features
Data/tool availability	High — public models via API, existing benchmarks as references	High — multiple model APIs, OSS benchmarks available	High — Docker, gVisor, Linux primitives all OSS; reference threat models from NIST/OWASP exist
Technical risk	Low-medium — graders can be tricky to make deterministic	Low-medium — depends on chosen benchmark's noisiness	Medium — the adversarial suite design <i>is</i> the project's novel contribution and requires the most original thought
Novelty / research signal	Medium — yet-another-benchmark unless task family is genuinely fresh	Medium — well-explored space (cost routing is an active product area)	Medium-high — open-source landscape has few rigorously-evaluated agent sandboxes with documented threat models
Fit with student's background	Excellent — pure applied SWE + API integration	Excellent — applied SWE + middleware	Good — applied SWE plus Linux/container deepening; the deepening is intentional growth
Portfolio signal for target role	Strong — "I can ship + evaluate agents"	Strong — "I think about agent infrastructure economics"	Strongest for the specific target (agent tooling / product SWE at Anthropic, DeepMind) — directly mirrors real product surfaces

Dimension	A. Coding-agent + benchmark	B. Multi-model router	C. Hardened sandbox
Map to CISC 699 rubric dimensions	Strong on evaluation; medium on technical-design rigor	Strong on technical design; medium on evaluation novelty	Strong on technical design <i>and</i> on evaluation rigor; strong on ethics/security/broader-impact
Worst-case failure mode	Benchmark too small or graders flaky → results lack power	Cost/quality numbers too noisy to draw conclusions	Scope creep into multi-language/persistence features → nothing finishes
Mitigation for worst-case	Tier the benchmark by difficulty; commit to a minimum-viable 30-task floor	Pre-register the benchmark choice + sample size; fix configurations before measurement	Hard scope-fence in charter; W7 midpoint review is the explicit stop-and-cut point

Selection: Candidate C

Why C over A and B (in the student's own future words; the below is rationale to revise):

- 1. Best portfolio fit for the stated career target.** The role family the student is targeting — agent tooling / product SWE at frontier labs — has "build the infrastructure agents run on top of" as one of its primary problem statements. C is a credible miniature of that work. A and B are both defensible but are one step removed from what those teams actually build day-to-day.
- 2. Strongest match to the rubric's evaluation criterion.** The CISC 699 RU-01 rubric weights "evaluation & evidence" heavily. C's central contribution *is* an evaluation methodology — a 40+ program adversarial benchmark plus an A/B comparison across isolation back-ends. The evaluation is harder to construct than A's or B's (which borrow from existing benchmarks), but precisely for that reason it scores higher on rubric levels that reward "demonstrates rigorous early feasibility across machine, architecture, computational method, and API/tool options; risks and dependencies are realistically anticipated."
- 3. Strong ethics/security/broader-impact story.** The rubric's "ethics & broader impact" dimension is a natural fit: prompt injection, agent misuse, dual-use considerations. C makes this dimension central rather than tacked-on.
- 4. Constraints honored.** The Python-only / no-network / no-GPU scope keeps the threat model and the timeline both honest. Hardware budget for cloud Linux VMs to run isolation experiments is well within the stated envelope. No model-training compute required.

Risks of choosing C, with mitigations

- Scope creep into multi-language or persistence features.** Mitigation: hard scope-fence in the charter, restated at W4 design review and W7 midpoint.
- Adversarial suite quality concerns.** A small or unrepresentative adversarial suite would undermine the central contribution. Mitigation: define category taxonomy in the design phase (W3–

W4), commit to ≥ 40 programs across ≥ 6 categories with rationale, peer-review by supervisor at midpoint.

- **Performance comparison requires careful statistics.** gVisor's overhead is well-known to be variable. Mitigation: define the benchmarking protocol upfront (warm-up runs, sample size, percentile reporting), draft confidence-interval analysis in design phase.
- **Linux kernel/host variability between dev and grading environments.** Mitigation: pin a single Ubuntu LTS image, document the environment, target Ubuntu 22.04 or 24.04 for both dev and grading.

What would change the decision

If, by end of W2, the supervisor signals that a sandbox project is too systems-heavy for the program's tolerance, the fallback is Candidate A — same time horizon, same compute budget, but a less systems-heavy artifact. Candidate B is the second fallback.